

Design and verification of a full-body control system for a drone equipped with a robotic arm based on GPU-MPPI

NPSK3

SEIoT MSc Final Project Report 2025

University College London

Supervisor: Valerio Modugno

ABSTRACT

This work focuses on the development of a custom quadrotor UAV equipped with a lightweight robotic arm, involving structural design, dynamic modeling, and a GPU-based whole-body control strategy. Firstly, an integrated mechanical design of the quadrotor and arm was completed. Based on this, a simulation pipeline was built using the MuJoCo physics engine and JAX-based numerical computation for controller validation. A GPU-parallelized Model Predictive Path Integral (MPPI) controller was then implemented via the JAX framework, enabling coordinated control of both the UAV and the manipulator. This allowed for trajectory tracking and precise end-effector positioning. To improve joint stability under PWM-based torque-like control, a novel PWM Deviation Damping (PWMDD) joint model was proposed, introducing a velocity deviation feedback mechanism for damping compensation. Finally, both simulation and physical experiments were conducted. In tracking tests, the MPPI controller achieved the lowest RMSE of **0.0386 m**, outperforming Random Shooting Model Predictive Control (RS-MPC) and Cross-Entropy Method MPC (CEM-MPC) (M=3) by over **20%**. Ablation studies confirm the proposed PWMDD model effectively suppresses joint oscillations. These results demonstrate improved stability and real-time feasibility for aerial manipulation and sampling tasks.

Author Keywords

Quadrotor UAV, Aerial Manipulator, Whole-Body Control, MPPI, JAX, MuJoCo Simulation, Trajectory Tracking, Real-Time Control.

Video Submission

github.com/Headmaster218/SkyGrip/tree/main/Medias

MSC SEIoT FINAL PROJECT REPORT

Project report submitted in part fulfilment of the requirements for the degree of Master of Science System Engineering for the Internet of Things, University College London, 2025.

NOTE BY THE UNIVERSITY

This project report is submitted as an examination paper. No responsibility can be held by University College London for the accuracy or completeness of the material therein.

1 INTRODUCTION

In the past decade, aerial manipulators, an emerging class of flying robots that combine the agility of unmanned aerial vehicles (UAVs) with the dexterous manipulation capabilities of robotic arms, have attracted significant attention from both the academic and engineering communities [30]. Compared to conventional UAVs used for payload transport or surveillance, UAVs equipped with manipulators can directly interact with the environment, enabling complex tasks such as grasping and manipulation.

However, UAVs are inherently underactuated and nonlinear systems, and the addition of one or more manipulators further increases the dynamic complexity and inter-component coupling of the overall system [54]. The motion of the manipulator causes time-varying changes in the UAV's center of mass, inertia, and external forces, which significantly affect attitude and position control. Conversely, UAV attitude corrections can interfere with the end-effector's positioning accuracy. This strong coupling renders traditional decoupled control approaches, where the UAV and manipulator are controlled separately, largely ineffective [16].

Moreover, to preserve flight performance, the manipulator must be highly lightweight, which in turn may lead to increased structural compliance and added control challenges. Ensuring stable and coordinated operation between the UAV and the manipulator thus remains a major challenge in the design of aerial manipulation systems.

To address the aforementioned challenges, considerable research has focused on the dynamic modeling and control of UAV-manipulator systems. A common traditional approach is the use of hierarchical control architectures: the UAV's position and attitude are stabilized by low-level controllers (e.g., PID or inner-loop stabilizers), while the manipulator joints are controlled independently [41]. To mitigate the mutual influence between the two subsystems, decoupling or compensation strategies are sometimes applied [19]. However, such methods often struggle to handle strong coupling effects and external disturbances.

Kim *et al.* developed a quadrotor prototype equipped with a two-degree-of-freedom manipulator and modeled the UAV and robotic arm as a unified dynamic system [12]. They implemented an adaptive sliding mode controller and successfully demonstrated autonomous aerial grasping and object delivery. Subsequent studies have further explored robust and adaptive control strategies for aerial manipulators, including attitude–arm coordination based on sliding mode control, backstepping, and disturbance observers, as well as hybrid force–position control schemes [56, 36, 18, 27, 22].

On the other hand, reinforcement learning and adaptive control methods have also been explored, but their reliability and safety remain below the threshold required for real-world deployment [4, 43, 53]. With the advancement of optimization algorithms and computational resources, MPC has increasingly been applied in UAV systems. One notable emerging variant is MPPI control [50].

MPPI is a sampling-based stochastic optimal control method that generates control inputs by drawing a large number of control trajectories under the system dynamics and computing a weighted average based on their cumulative cost [50]. Unlike conventional MPC approaches that rely on gradient information or local linearization, MPPI does not require an explicit linear model and is capable of handling nonlinear dynamics and complex constraints directly.

Williams *et al.* proposed a GPU-accelerated implementation of MPPI, enabling real-time control in systems such as autonomous vehicles [48]. Given these advantages, applying MPPI to the whole-body control of aerial manipulators offers a promising approach to managing the nonlinear and tightly coupled dynamics between the UAV platform and its robotic arm.

This paper investigates the dynamic modeling and whole-body control of a custom-developed quadrotor UAV equipped with a robotic manipulator (see Fig. 1). Unlike previous works, a GPU-parallelized MPPI control algorithm is adopted to achieve unified optimal control of both the UAV and the manipulator. Additionally, to address the characteristics of PWM-based servo actuation, a deviation damping model was proposed to improve system stability.

The main contributions of this work are as follows:

- Design of a quadrotor UAV system with a two-DoF robotic arm, completing the full development cycle from mechanical design and component selection to dynamic model identification.
- Development of a simulation and control platform integrating PhaseSpace, MuJoCo, `sbmpc` [1], and JAX, enabling a closed-loop pipeline from phys-

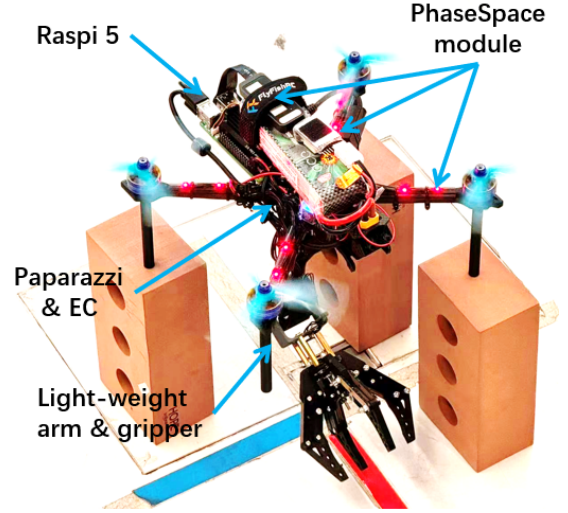


Figure 1: Custom-developed aerial manipulator platform consisting of a quadrotor UAV with a 2-DoF robotic arm.

ical parameter acquisition to real-time optimal control.

- Implementation of a JAX-accelerated MPPI whole-body controller that coordinates UAV position, attitude, and manipulator end-effector control under complex system dynamics.
- Proposal of a PWMDD joint model that introduces servo velocity feedback to enhance consistency between simulation and real-world behavior under PWM torque-like control.
- Validation of the proposed control framework through both comparative simulations and real-world experiments.

The remainder of this paper is organized as follows: Section 2 reviews related work and current research. Section 3 details the system design, dynamic modeling, and the implementation of a unified whole-body control strategy based on MPPI. Section 4 provides detailed parameters, results, and analysis of both simulation and real-world experiments. Section 5 concludes the paper and discusses directions for future work.

2 LITERATURE REVIEW

Robust and precise control of aerial manipulator systems remains a significant challenge due to the strong coupling between the UAV platform and the onboard robotic arm. To address this, a wide range of research efforts have focused on both the mechanical design of aerial manipulation platforms and the development of suitable dynamic models and control strategies. By examining prior work, the review reveals both the advantages and limitations of existing methods, thereby

motivating the adoption of a GPU-accelerated MPPI-based whole-body control approach in this study.

2.1 Platform Design

The hardware design of aerial manipulation robots requires a careful balance between flight performance and manipulation capability. Early studies often employed simple grippers or magnetic attachments suspended beneath small UAVs to perform object pickup tasks [29]. These designs are structurally simple and lightweight, but the orientation of the end-effector is entirely constrained by the UAV’s attitude, offering only 4 degrees of freedom (3 positional + yaw angle).

An alternative approach involves suspending the payload using flexible cables, allowing the UAV to indirectly guide the payload’s position through its motion. While this cable-suspended method avoids the complexity of a robotic arm, it lacks active control over the payload’s orientation and suffers from uncontrollable oscillations [13, 14].

To achieve more dexterous manipulation, researchers began exploring the direct integration of multi-DoF manipulators onto UAV platforms. For example, Korpela et al. mounted three two-joint robotic arms beneath a UAV, creating a “flying spider” prototype capable of multi-point coordinated manipulation [21]. However, this system proved overly complex, and the arm mass consumed most of the UAV’s payload capacity.

In contrast, single-arm configurations are more common. Kim et al. [20] presented an aerial manipulation system using a 2-DoF arm, with its joint axes were parallel, limiting the orientation workspace. In the work by Fanni and Khalifa [10], a two-joint manipulator was mounted along the UAV’s central bottom axis, with orthogonal joint axes enabling two independent rotational DoF for the end-effector in the UAV body frame. This configuration allows the manipulator to execute small push-pull motions in arbitrary directions. The arm used high torque-density micro servo actuators and hollow carbon fiber links. Finite Element Analysis (FEA) showed that the structure had sufficient stiffness to withstand the maximum end-effector forces, while the total mass was kept under 0.2 kg. The lightweight design minimized the impact of arm motion on UAV attitude, allowing for stable hovering during manipulation. The manipulator’s motion range was carefully constrained based on UAV size and mission requirements to ensure end-effector reachability within the typical operational envelope.

2.2 Control Strategies

When a UAV is equipped with a robotic manipulator, its dynamic model becomes significantly more com-

plex than that of a standalone UAV. Specifically, the coupled motion between the UAV’s rigid body and the manipulator chain must be accurately described. As early as 2011, Michael et al. developed an Euler–Lagrange dynamic model for a quadrotor with an onboard manipulator, highlighting how manipulator motion induces attitude coupling and analyzing the resulting reaction forces on the UAV [34].

Subsequent studies have used this model to design controllers or observers that compensate for such coupling effects. For instance, disturbance observers (DOb) have been employed to estimate the forces exerted by the manipulator in real-time and provide feedback corrections to the UAV control commands, thereby maintaining flight stability [23, 6, 40, 8].

In classical control architectures, UAV attitude and altitude control are typically treated as the inner loop, while manipulator joint or end-effector control forms the outer loop. The inner loop, handled by the UAV’s flight controller, ensures stable hovering, while the outer loop plans the manipulator’s motions according to task requirements, taking the UAV’s motion into account [24, 11].

However, when the manipulator has considerable mass or executes rapid movements, its influence on the UAV becomes non-negligible. In such cases, conventional cascaded control is insufficient for timely compensation, and a more tightly coupled whole-body control strategy becomes necessary [38, 44].

2.2.1 Whole-Body Control

The concept of whole-body control, which treats the base and manipulator as a single integrated system for control optimization, has been widely applied in mobile manipulators and legged robots. In the context of UAV–manipulator systems, one practical implementation is Nonlinear Model Predictive Control (NMPC) [15, 17, 32, 9, 5, 47]. NMPC solves a finite-horizon optimal control problem online at each control step, producing a sequence of future control inputs that explicitly account for system dynamics and state/input constraints [7, 15].

Bouffieux *et al.* (representative NMPC implementations on quadrotors with manipulator or strong coupling) report that NMPC outperforms classical PID under strong coupling, but the key limitation is computational cost since each step solves a high-dimensional nonlinear program; this makes high-rate control challenging on embedded platforms [17, 5, 32, 9]. To mitigate this, model reduction or decomposed strategies are often adopted, e.g., simplifying the UAV to a point-mass, or retaining only slow manipulator dynamics, but such simplifications may degrade accuracy and robustness [15, 39]. Recent work pushes real-time

NMPC via tailored solvers and parallelization, including multilevel GPU-accelerated SQP for NMPC [46], as well as efficient embedded frameworks such as acados and code-generation pipelines [47]. Complementary developments on flight-control stacks and middleware (PX4/ROS 2) and JAX-based differentiable computation further reduce integration and prototyping cost [28, 25, 3].

As an alternative to gradient-based NMPC, sampling-based stochastic optimal control has gained traction. RS-MPC evaluates many sampled control sequences by forward simulating trajectories and selecting the best; it is gradient-free and highly parallelizable but can be sample-inefficient [52]. The Cross-Entropy Method (CEM) iteratively updates a sampling distribution using elite samples to improve efficiency, yet may suffer from local minima and added iteration cost [37]. MPPI control provides a principled, importance-sampling-based formulation with strong GPU affinity; it has been demonstrated in aggressive driving and generalized to parallel stochastic optimization on modern GPUs [51, 52, 49]. Extensions combine MPPI with robust/adaptive augmentation (e.g., $\mathcal{L}_1$ adaptive MPPI) and show competitive performance on multirotors [35, 31, 45].

In whole-body aerial manipulation, where manipulator mass and fast motion induce significant reaction on the UAV, tightly coupled controllers (either NMPC with accurate constraints or massively parallel sampling-based controllers) are preferable to cascaded designs. Geometric control and flatness results remain valuable for trajectory generation (e.g., cable-suspended loads) and can serve as warm-starts or structural priors for MPC/MPPI formulations [42, 26]. Finally, practical deployments must consider communication delays, EMI/cybersecurity, and onboard compute limits, factors that influence solver choice and dictate GPU-oriented implementations on embedded platforms [5, 55, 2, 33].

2.2.2 Model Predictive Path Integral

MPPI is a sampling-based control algorithm that combines stochastic optimal control with importance sampling theory. It was first introduced by Williams *et al.* in the context of autonomous driving control [52, 51]. Unlike methods that select only the single best sampled trajectory, MPPI computes a weighted average of control sequences based on the cumulative cost of each trajectory, and uses this average to update the control input increment. This mechanism is rooted in the path integral formulation, where the optimal control problem is reformulated as an expectation over stochastic trajectory contributions, hence the method is also referred to as Information-Theoretic MPC [51].

MPPI eliminates the need to compute gradients of the cost function, making it naturally suitable for irregular cost landscapes and high-dimensional action spaces. Additionally, by softly aggregating information from all samples, MPPI improves sampling efficiency and reduces sensitivity to noise. Studies have shown that under the same number of samples, MPPI achieves smoother control outputs and better robustness compared to Random Shooting and CEM methods [35].

The main drawback of MPPI lies in its computational demand: real-time control of high-speed systems often requires evaluating tens of thousands of trajectories per second. Fortunately, the MPPI computation process, parallel simulation of trajectories and weighted aggregation, is well-suited to GPU-based parallel architectures. Recent efforts such as MPPI-Generic [49] provide CUDA-optimized implementations for stochastic optimization.

In a recent 2024 study, Minařík *et al.* deployed MPPI on an onboard GPU of a UAV and achieved real-time obstacle avoidance at speeds up to 44 km/h [31]. They utilized JAX with Just-In-Time (JIT) compilation to accelerate MPPI, converting Python-based algorithms into optimized XLA machine code [3]. This enabled embedded platforms to reach kilohertz-level sampling frequencies. Another comparative study evaluated acados-based NMPC [47] versus JAX-accelerated MPPI on NVIDIA Jetson platforms and found that MPPI achieved similar trajectory tracking accuracy while being an order of magnitude faster in computation and more straightforward to tune. These results suggest that MPPI, when combined with GPU acceleration, offers a promising solution for real-time control in complex robotic systems.

2.3 Summary and Research Gap

In summary, aerial manipulation has progressed from passive payload handling to integrated UAV-manipulator platforms, with growing focus on dynamic modeling and control. While classical cascaded controllers remain common, their limitations under strong coupling and rapid manipulator motions have prompted the development of whole-body control strategies such as NMPC and sampling-based MPC. These offer better performance but face challenges in computational cost, real-time feasibility, and actuator robustness.

Among these, Model Predictive Path Integral (MPPI) control stands out for its gradient-free formulation and GPU compatibility. However, its deployment in physical aerial manipulation systems remains limited. In particular, the integration of MPPI with low-cost PWM actuators, embedded GPU platforms,

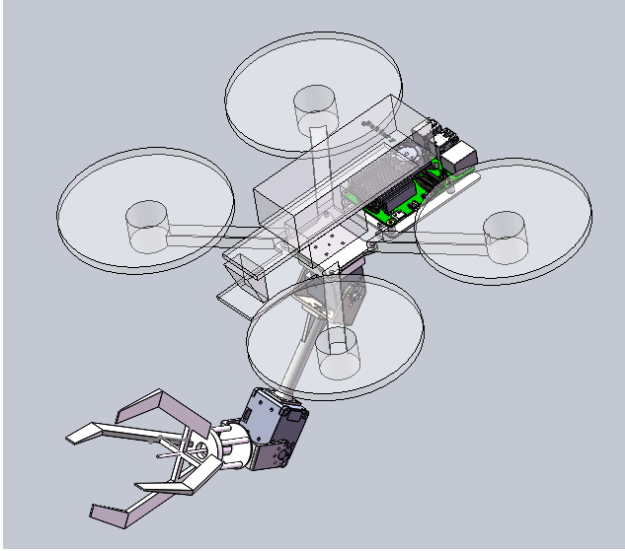


Figure 2: The mechanical Structure of UAV with Arm in Solidworks

and system delay compensation is underexplored. This research addresses these gaps by implementing a real-time MPPI controller on a lightweight aerial manipulator, with explicit modeling of actuator dynamics and control latency.

3 SYSTEM DESIGN AND METHODOLOGY

This section presents the overall design of the autonomous quadrotor-manipulator system, including the mechanical platform, derivation of the dynamic model, the development of an MPPI controller based on the dynamic model [1], and the proposed PWMDD method for servo dynamics model.

3.1 Structure and Component

As illustrated in Fig. 2, the system consists of a quadrotor UAV base and a two-freedom robotic arm mounted 0.05 m below its center of mass. The quadrotor adopts an “X” configuration with a lightweight yet robust carbon-fiber composite frame and integrates a flight controller, power battery, Raspberry Pi, and other modules, yielding a total takeoff weight of approximately 1.2 kg.

The robotic arm comprises two serial joints with link lengths of about 0.13 m and 0.16 m, respectively. A simple gripper or task-specific tool can be attached at the end-effector. This reflects the high portability of the modular approach to mechanical design. The first joint rotates about an axis parallel to the UAV body’s y-axis (pitch axis), while the second joint rotates about a parallel axis, thereby enabling motion within a vertical plane.

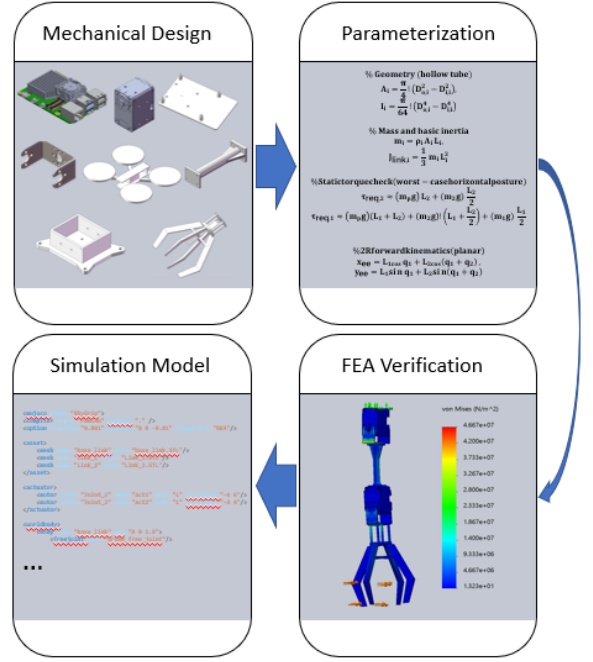


Figure 3: Design and modeling workflow for the aerial manipulation system.

This configuration leverages the UAV’s inherent agility and, when combined with whole-body control strategies, achieves a large operational workspace while maintaining a compact and structurally efficient design.

The four rotor motors are selected based on the total system weight and the expected additional load from arm movements. Each motor-propeller pair provides a maximum static thrust of ≈ 10 N and a rated thrust of ≈ 6 N, sufficient to compensate for center-of-mass shifts caused by manipulator motion and external payloads.

The robotic arm joints are actuated by smart servo motors, such as the Dynamixel XM series. The first joint provides a peak torque of approximately 3.0 N·m, and the second joint about 1.5 N·m, with corresponding no-load speeds of ≈ 77 rad/s and ≈ 57 rad/s, respectively. The servos support multiple control modes, including position, PWM, and velocity control. Built-in position and velocity sensors enable closed-loop control and real-time state feedback.

3.2 Simulation and Control Methodology

To facilitate the development and testing of control algorithms, an integrated methodology was established, spanning from CAD-based modeling to physics-based simulation. The UAV and manipulator were first modeled in SolidWorks, and the corresponding URDF (Unified Robot Description Format) file was exported to extract accurate mass, inertia, and geometric pa-

rameters for each component.

The URDF model was then imported into the MuJoCo physics engine for high-fidelity simulation. MuJoCo provides precise rigid-body dynamics and collision detection, enabling realistic physical interactions. Parallely, the corresponding dynamics functions were implemented in Python and accelerated using the JAX library to support fast, batched trajectory simulations required in MPPI control.

The entire simulation environment supports hardware-in-the-loop synchronization. By reading real-time joint angles from the servos and pose data from UAV sensors, the current physical state can be injected into the simulation to maintain consistency between the digital twin and the real system. The control outputs (forces and torques) can be used either to advance the simulation or to be transmitted via a data link to the actual ESCs and servo actuators.

The complete workflow from mechanical design to simulation integration is illustrated in Fig. 3.

During hardware experiments, control torques are applied directly to the UAV, while the UAV’s real-time state is fed back to the controller and reflected in the MuJoCo visualization interface. This simulation–reality synchronization mechanism ensures the transferability and robustness of the proposed control algorithms in real-world deployment.

3.3 Dynamics and Actuation Modeling

This section presents a comprehensive formulation of the system dynamics and actuator modeling for the aerial manipulator, encompassing the UAV–manipulator coupling, force and moment equations, and the mapping between control inputs and motor-level actuation.

3.3.1 Dynamic Model of UAV–Manipulator System

The full dynamic model of the quadrotor-manipulator system captures the coupled translational, rotational, and joint dynamics under the influence of external forces, internal actuation, and inter-body interactions.

The system state vector $\mathbf{x} \in \mathbb{R}^{18}$ includes the UAV position $\mathbf{p} = [x, y, z]^T$, as shown in Fig. 4, orientation quaternion $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$, joint angles q_1, q_2 , linear velocity $\mathbf{v} = [v_x, v_y, v_z]^T$, angular velocity $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$, and joint angular velocities $\dot{q}_1, \dot{q}_2$:

$$\mathbf{x} = [\mathbf{p}, \mathbf{q}, q_1, q_2, \mathbf{v}, \boldsymbol{\omega}, \dot{q}_1, \dot{q}_2]^T \quad (1)$$

The control input vector $\mathbf{u} \in \mathbb{R}^6$ comprises the total thrust T , body torques $\tau_\phi, \tau_\theta, \tau_\psi$, and joint torques τ_1, τ_2 :

$$\mathbf{u} = [T, \tau_\phi, \tau_\theta, \tau_\psi, \tau_1, \tau_2]^T \quad (2)$$

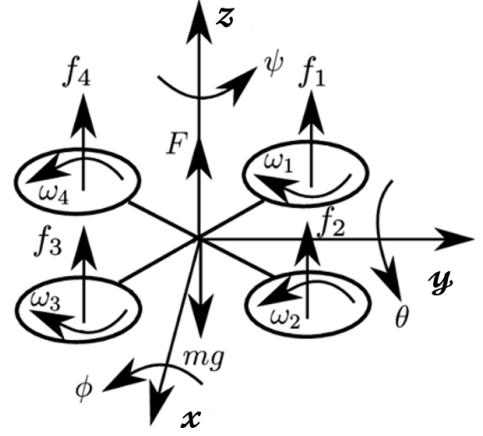


Figure 4: Dynamic model of the aerial manipulator system in the body-fixed frame.

System Dynamics:

The unified dynamics of the aerial manipulator are governed by the following coupled equations, capturing both translational and rotational motion of the UAV body, as well as the internal dynamics of the manipulator joints.

$$M_{\text{total}} \dot{\mathbf{v}} = T \mathbf{R}(\mathbf{q}) \hat{\mathbf{z}} + M_{\text{total}} \mathbf{g} - c_v \mathbf{v} \quad (3)$$

$$\mathbf{I}_b \dot{\boldsymbol{\omega}} = \boldsymbol{\tau}_b + \boldsymbol{\tau}_{\text{arm}} + \boldsymbol{\tau}_{\text{com}} - \boldsymbol{\omega} \times (\mathbf{I}_b \boldsymbol{\omega}) - c_\omega \boldsymbol{\omega} \quad (4)$$

$$\boldsymbol{\tau}_{\text{arm}} = \begin{bmatrix} 0.002\dot{q}_1 + 0.001\dot{q}_1\dot{q}_2 \\ 0.002\dot{q}_2 + 0.001\dot{q}_1\dot{q}_2 \\ 0.001(\dot{q}_1^2 + \dot{q}_2^2) \end{bmatrix} \quad (5)$$

$$\boldsymbol{\tau}_{\text{com}} = \Delta \mathbf{r}_c \times (T \cdot \mathbf{R}(\mathbf{q}) \hat{\mathbf{z}}) \quad (6)$$

$$I_j \ddot{q}_j = \tau_j - D_j \dot{q}_j - F_j \text{sgn}(\dot{q}_j) - \tau_G(q_j) \quad (7)$$

The complete dynamics of the aerial manipulator system consist of translational, rotational, attitude, and joint-level components. The translational dynamics describe the UAV’s center-of-mass motion under the influence of thrust, gravity, and linear aerodynamic drag, where M_{total} is the total system mass, $\mathbf{g} = [0, 0, -9.81]^T \text{ m/s}^2$ is the gravitational acceleration, c_v is the linear damping coefficient, and $\hat{\mathbf{z}}$ is the unit vector along the body-frame thrust direction. The rotational dynamics incorporate the UAV’s angular inertia, actuator torques, manipulator-induced disturbances, and angular damping, where $\boldsymbol{\tau}_b = [\tau_\phi, \tau_\theta, \tau_\psi]^T$ are the body torques, $\mathbf{I}_b$ is the UAV inertia matrix, and c_ω is the angular damping coefficient. The manipulator contributes disturbance torques, including internal motion-induced terms $\boldsymbol{\tau}_{\text{arm}}$ and center-of-mass shift effects, where $\Delta \mathbf{r}_c$ is the offset between the instantaneous system center of mass and the UAV body frame. Attitude kinematics are updated via quaternion propagation using $\dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}) \mathbf{q}$. At the joint level, each ac-

tuator obeys a second-order dynamic model, where I_j is the joint inertia, D_j and F_j are viscous and Coulomb damping coefficients, and $\tau_G(q_j)$ denotes gravitational compensation.

3.3.2 Thrust and Torque Mapping for Quadrotor Inputs

To implement low-level control on a quadrotor UAV, the controller's desired inputs, total thrust T and body torques $\tau_b = [\tau_\phi, \tau_\theta, \tau_\psi]^T$, must be converted into individual rotor speeds $\omega = [\omega_1, \omega_2, \omega_3, \omega_4]^T$. Each rotor generates an upward thrust force $f_i = k_f \omega_i^2$ and a reaction torque $d_i = (-1)^i k_m \omega_i^2$, where k_f is the thrust coefficient and k_m is the moment (drag) coefficient.

As shown in Figure 4, the four rotors are arranged in an 'X'-shaped configuration, and their collective effects generate the overall control inputs as:

$$\begin{bmatrix} T \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \underbrace{\begin{bmatrix} k_f & k_f & k_f & k_f \\ 0 & -k_f l & 0 & k_f l \\ -k_f l & 0 & k_f l & 0 \\ k_m & -k_m & k_m & -k_m \end{bmatrix}}_A \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} \quad (8)$$

Here, l denotes the distance from the UAV center to each rotor, and A is the allocation matrix mapping squared rotor speeds to the UAV's net thrust and torques.

Given a desired control input $\mathbf{u}_{\text{UAV}} = [T, \tau_\phi, \tau_\theta, \tau_\psi]^T$, the corresponding squared rotor speeds are computed by inverting the allocation matrix:

$$\omega^2 = A^{-1} \mathbf{u}_{\text{UAV}} \quad (9)$$

To ensure physical feasibility, the resulting ω_i^2 values are clipped to fall within the actuator's operational limits $[\omega_{\min}^2, \omega_{\max}^2]$. The computed squared speeds are then passed to the motor driver, which converts them to PWM duty cycles accordingly.

This thrust-to-rotor mapping bridges the high-level control design and low-level actuator commands, enabling consistent torque generation during aggressive maneuvers.

3.3.3 Servo Compensation Model (PWMDD)

In conventional joint models, it is often assumed that $\tau_{\text{eff}} \approx \tau_{\text{cmd}}$, i.e., the motor can directly deliver the commanded torque. However, under PWM voltage control, the servo operates in an open-loop manner: the controller specifies only a duty cycle (PWM) without closed-loop speed regulation. In practice, the PWM duty ratio is interpreted as the desired no-load angular velocity ω_{pwm} , which is then scaled by a constant gain

to form the commanded torque, $\tau_{\text{cmd}} = k_{\tau, \text{PWM}} \cdot \text{PWM}$. When the joint operates under load, the actual angular velocity ω_{act} becomes lower than ω_{pwm} . The resulting speed error

$$\Delta\omega = \omega_{\text{pwm}} - \omega_{\text{act}} \quad (10)$$

induces additional current in the H-bridge, effectively producing a braking torque. For simplicity, this effect is modeled as a virtual damping term proportional to $\Delta\omega$, yielding the actual effective torque:

$$\tau_{\text{eff}} = \tau_{\text{cmd}} + \tau_{\text{stall}} \cdot \frac{\Delta\omega}{\omega_{\text{no-load}}} \quad (11)$$

where τ_{stall} is the stall torque and $\omega_{\text{no-load}}$ is the maximum no-load angular velocity, both obtainable from datasheets or test-bench experiments.

The behavior of function 11 can be summarized as follows:

If $\omega_{\text{act}} < \omega_{\text{pwm}}$ (load slows the motor), the second term is positive, providing compensatory torque. If $\omega_{\text{act}} > \omega_{\text{pwm}}$ (joint overspeed due to inertia or external forces), the second term is negative, acting as reverse braking to suppress overshoot. If $\text{PWM} = 0$ while the joint continues to rotate due to inertia, then $\omega_{\text{pwm}} = 0$, and function 11 reduces to a pure damping torque, emulating the holding-brake effect when the servo is unpowered.

To prevent overestimation of braking torque at high speeds, τ_{eff} is bounded by:

$$\tau_{\min} \leq \tau_{\text{eff}} \leq \tau_{\max}, \quad \text{with } \text{sign}(\tau_{\text{eff}}) = \text{sign}(\Delta\omega) \quad (12)$$

Substituting τ_{eff} into the joint dynamics??, together with viscous damping and friction terms, yields the joint motion equations. The gravitational torque τ_G is additionally incorporated, for example in the case of the two-link arm considered here:

$$\begin{aligned} \tau_{G1} &= -(M_{L1} l_{1c} + M_{L2} l_2) g \sin q_1, \\ \tau_{G2} &= -M_{L2} l_{2c} g \sin(q_1 + q_2) \end{aligned} \quad (13)$$

where M_{L1}, M_{L2} are the link masses and l_{1c}, l_{2c} denote the distances from joint axes to the link centers of mass. Near-zero joint angles, the gravitational term is smoothed to avoid singularities and chattering.

Finally, solving the coupled system of equations Eq. 7 provides the complete set of differential equations governing the time evolution of the UAV-manipulator system.

3.4 Whole-Body Control Framework

This section introduces the whole-body control framework adopted in this work, which unifies the control of UAV dynamics and manipulator motion under a single predictive optimization scheme.

3.4.1 Whole-Body Controller Design Based on MPPI

Building on the established system dynamics model, a whole-body optimal controller is designed using the Model Predictive Path Integral (MPPI) algorithm to compute the joint control inputs of the quadrotor-manipulator system.

At the current time step $t = 0$, with a prediction horizon of length H , the baseline control sequence is defined as

$$\boldsymbol{\mu} = \{\boldsymbol{\mu}_0, \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_{H-1}\}, \quad \boldsymbol{\mu}_t \in \mathbb{R}^6 \quad (14)$$

where each $\boldsymbol{\mu}_t$ corresponds to

$$\boldsymbol{\mu}_t = [T, \tau_\phi, \tau_\theta, \tau_\psi, \tau_1, \tau_2]^\top \quad (15)$$

For the k -th sampled trajectory, random control perturbations are applied to $\boldsymbol{\mu}$, and the system dynamics are rolled out to generate the corresponding state sequence

$$\left\{ \mathbf{x}_t^{(k)} \right\}_{t=0}^H \quad (16)$$

The cumulative cost is computed as

$$J^{(k)} = \sum_{t=0}^{H-1} L(\mathbf{x}_t^{(k)}, \mathbf{u}_t^{(k)}) + \Phi(\mathbf{x}_H^{(k)}) \quad (17)$$

where L is the stage cost, incorporating position error, velocity, angular velocity, tilt angle, joint velocity, control regularization, and soft constraints, while Φ denotes the terminal cost.

Each sampled trajectory is assigned a weight according to the path integral formulation:

$$w^{(k)} = \frac{\exp\left(-\frac{1}{\lambda} J^{(k)}\right)}{\sum_j \exp\left(-\frac{1}{\lambda} J^{(j)}\right)} \quad (18)$$

where λ is a temperature parameter that balances exploration and exploitation. The updated control sequence is then obtained as the weighted average:

$$\boldsymbol{\mu}_t \leftarrow \sum_k w^{(k)} \mathbf{u}_t^{(k)}, \quad t = 0, \dots, H-1 \quad (19)$$

By construction, $\sum_k w^{(k)} = 1$, ensuring the update is a convex combination.

Only the first control input:

$$\mathbf{u}^* = \boldsymbol{\mu}_0 \quad (20)$$

is applied to the quadrotor-manipulator at the current control step. The remaining control sequence is shifted forward one step (receding horizon) and used as the baseline for the next iteration. Thus, the cycle of sampling $\rightarrow$ rollout $\rightarrow$ cost evaluation $\rightarrow$ exponential weighting $\rightarrow$ control update achieves stochastic optimization over the H -step horizon while preserving smoothness and exploration capability in high-dimensional control spaces.

3.4.2 Loss Function Design

The overall control objective is to achieve unified coordination between the UAV and its onboard manipulator under dynamic conditions. Specifically, the system is required to accurately track a desired 3D position trajectory for the UAV's center of mass, while maintaining stable attitude regulation to ensure flight stability and directional control. Concurrently, the manipulator must either follow a prescribed end-effector path or execute specific joint configurations, depending on the task context (e.g., grasping, placing, or pointing). Throughout the control process, the framework must also penalize aggressive maneuvers and suppress excessive actuator usage to ensure smooth motion, minimize energy consumption, and prevent mechanical wear, particularly under long-horizon operation and frequent perturbations.

To this end, a structured lossfunction is designed to capture the various objectives and constraints required for real-time operation. The core idea is to minimize a composite cost function over the prediction horizon that simultaneously guides the UAV's flight behavior and the manipulator's joint dynamics, while penalizing unstable or aggressive motions. The overall loss is defined as follows:

$$\begin{aligned} \ell = & w_{\text{trk}} \|\mathbf{x}_{\text{ee}} - \mathbf{x}_{\text{ref}}\|^2 + w_v \|\mathbf{v}\|^2 \\ & + w_\omega \|\boldsymbol{\omega}\|^2 + w_{\text{tilt}} \cdot (\text{tilt})^2 + w_q \|\dot{\mathbf{q}}_{\text{arm}}\|^2 \\ & + \rho_T \cdot (\Delta T)^2 + \rho_\tau \|\boldsymbol{\tau}_{\text{uav}}\|^2 + \rho_{\tau_a} \|\boldsymbol{\tau}_{\text{arm}}\|^2 \\ & + \kappa ([q_1 - q_{1,\max}]_+^2 + [q_2 - q_{2,\max}]_+^2) \end{aligned} \quad (21)$$

Each term in the loss function corresponds to a specific physical or control-related objective, described in detail below.

Position and End-Effector Tracking.

The most critical term in the cost function is the squared Euclidean distance between the reference and actual positions, either for the UAV center of mass or the manipulator end-effector, depending on the task. For end-effector tracking, the position error is computed as $e_{\text{ee}} = \|\mathbf{p}_{\text{ee}} - \mathbf{p}_{\text{ref}}\|^2$, whereas for UAV trajectory tracking, the cost becomes $e_{\text{pos}} = \|\mathbf{p} - \mathbf{p}_{\text{ref}}\|^2$. A large weight (e.g., $w_{\text{trk}} = 300$) is applied to prioritize accurate goal-reaching behavior.

Velocity Regularization.

To ensure smooth motion and suppress undesired oscillations, both translational and rotational velocities are penalized. The UAV linear velocity norm $\|\mathbf{v}\|^2$ and angular velocity norm $\|\boldsymbol{\omega}\|^2$ are included with moderate weights (typically $w_v, w_\omega \approx 3.0$). These terms prevent excessive motion or instability during flight.

Tilt Suppression.

To prevent unsafe body inclination, a tilt penalty is

added using the metric $\text{tilt} = 1 - R_{zz}$, where R_{zz} is the bottom-right element of the rotation matrix $\mathbf{R}$, representing the alignment of the body z -axis with gravity. A weight $w_{\text{tilt}} \approx 12.0$ is assigned to encourage level and stable flight.

Joint Velocity Minimization.

To reduce mechanical stress and suppress oscillations, the squared joint angular velocity norm $\|\dot{\mathbf{q}}_{\text{arm}}\|^2$ is penalized. This term helps ensure smooth arm motion, particularly when interacting with soft or dynamic targets. A typical weight is $w_q \approx 7.5$.

Control Effort Regularization.

To reduce actuator energy usage and avoid saturation, the loss includes multiple regularization terms:

Thrust deviation: $\Delta T = T - M_{\text{total}}g$, penalized by $\rho_T \approx 0.15$.

UAV torque norm: $\|\boldsymbol{\tau}_{\phi, \theta, \psi}\|^2$, penalized by $\rho_\tau \approx 0.4$.

Manipulator torque norm: $\|\boldsymbol{\tau}_{1,2}\|^2$, penalized by $\rho_{\tau_a} \approx 5.0$.

These help avoid over-aggressive maneuvers and reduce wear on hardware components.

Joint Limit Constraints.

To softly enforce physical joint angle bounds (e.g., ± 1.3 rad), a hinge loss is added when any joint exceeds its limits. For joint i , the penalty is defined as

$$\kappa \cdot ([q_i - q_{i,\max}]_+^2),$$

where $[x]_+ = \max(0, x)$ denotes the ReLU operator. A large penalty weight (e.g., $\kappa = 1000$) ensures strict adherence to joint range limits.

Total Trajectory Cost and MPPI Update.

The cumulative trajectory cost J is computed by summing all stage costs ℓ over the prediction horizon, with an optional terminal cost Φ applied at $t = H$. In each control step, MPPI samples N candidate control sequences $\{\mathbf{u}_0^{(i)}, \dots, \mathbf{u}_{H-1}^{(i)}\}$, computes the cost $J^{(i)}$, and forms a weighted average of control inputs:

$$\bar{\mathbf{u}} = \frac{\sum_i w_i \mathbf{u}^{(i)}}{\sum_i w_i} \quad (22)$$

The weights w_i are computed by exponentiating the negative cost $J^{(i)}$. The first control input $\bar{\mathbf{u}}_0$ is applied to the system, while the control sequence is shifted for the next timestep. This enables real-time, whole-body optimal control under dynamic constraints and disturbances.

This section introduced the full-stack design of the aerial manipulation system, encompassing both hardware and algorithmic components. The section began with the mechanical design of a lightweight quadrotor-manipulator platform and its integration with external sensing and onboard computing units. A comprehensive dynamic model was derived to account for key factors such as center-of-mass shifts, pro-

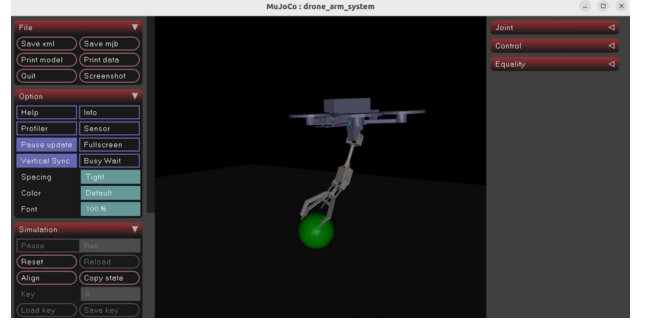


Figure 5: MuJoCo simulation environment, visualizing the quadrotor-manipulator system performing goal task.

PELLER-torque mapping, and manipulator coupling effects. Building upon this model, a GPU-accelerated MPPI controller was developed to generate whole-body optimal control commands in real-time. Furthermore, a PWM Deviation Damping (PWMDD) model was proposed to bridge the gap between simulated and real actuator behavior, especially under open-loop PWM control. These foundational components establish a unified control architecture for subsequent simulation and real-world experiments.

4 EXPERIMENTS AND RESULTS

To validate the effectiveness of the proposed method, both simulation-based evaluations and real-world experiments were conducted. This section is organized into two main sections: simulation-based evaluation and physical platform testing, with corresponding results and analyses presented sequentially.

4.1 Simulation Experiments

To comprehensively assess the performance of the proposed controller, a series of simulation experiments were carried out within a high-fidelity physical simulation environment built on the MuJoCo engine and a JAX-based computational backend, as shown in Fig. 5.. The simulation model is based on the custom-designed quadrotor-manipulator platform, with dynamic parameters and control structure kept consistent with the real hardware system.

The MPPI controller is evaluated from the following three perspectives:

- i) Comparison with baseline controllers (e.g., CEM-MPC, RS-MPC);
- ii) Sensitivity analysis of MPPI internal parameters (prediction horizon H , number of samples N , ablation of cost terms);
- iii) System recovery capability under external disturbances.

All experiment groups were conducted under the same simulation environment, control frequency, and task settings. The control loop runs at 55 Hz, with JAX-based parallel sampling configured with $N = 5000$ samples per step and prediction horizon $H = 80$. The simulation uses a time step of 18 ms and a receding horizon (sliding window) update strategy for the control sequence.

4.1.1 Control Algorithm Comparison: MPPI vs CEM-MPC vs RS-MPC

Task Background: Multi-waypoint trajectory tracking is a core task in aerial manipulation, requiring high-precision control of complex paths during high-speed flight. Due to strong nonlinear coupling between the UAV and manipulator, as well as the high-dimensional state space, traditional linear controllers (e.g., PID or LQR) often suffer from instability and error accumulation in such scenarios.

To address these challenges, this section compares the performance of three representative sampling-based predictive control methods:

- a) **MPPI (Model Predictive Path Integral):** Based on path integral theory, MPPI generates control updates by computing a softmax-weighted average over a large number of sampled trajectories. This yields smooth and robust control performance.
- b) **CEM-MPC (Cross Entropy Method):** CEM selects an elite subset of top-performing trajectories and uses their mean as the update direction. It is known for fast convergence and strong stability.
- c) **Random Shooting (RS-MPC):** Also known as Greedy Shooting MPC, this method simply chooses the single lowest-cost trajectory at each timestep. It offers high computational efficiency but tends to suffer from reduced control stability.

Experimental Setup: The UAV is tasked with following a tilted, closed square trajectory with edge length of 1 m, starting and ending at the same point. The initial condition places the UAV in hover at the starting location, with the manipulator joints initialized at zero.

All three controllers, MPPI, CEM-MPC, and RS-MPC, share the same system dynamics model and cost function; the key differences lie in how each method updates the control sequence. To ensure fairness in comparison, all controllers adopt an identical cost configuration. For CEM-MPC, the number of elite samples M is tested at values of 3, 7, and 30, while the

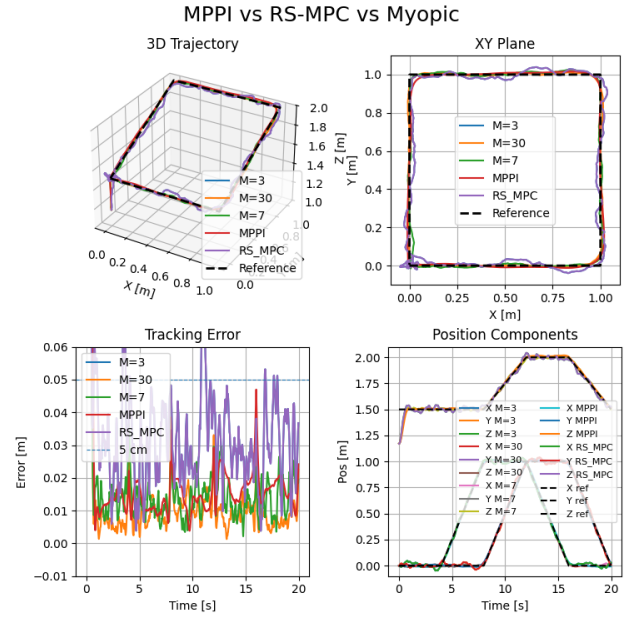


Figure 6: Comparison of tracking trajectories and errors under MPPI, CEM-MPC ($M = 3, 7, 30$), and RS-MPC.

MPPI temperature parameter λ is tuned to approximately 0.7 to balance exploration and loss scaling.

Result and Analysis: Figure 6 compares the tracking performance of MPPI, RS-MPC, and CEM-MPC with different elite set sizes ($M = 3, 7, 30$) on the cubic trajectory task.

The 3D trajectory and XY plane projection sub-figures of Figure 6 (top two) illustrate the spatial execution of the planned path under the three control strategies. It can be observed that both MPPI and CEM-MPC closely follow the reference trajectory in all directions with smooth transitions. In contrast, RS-MPC shows significant overshoot and jitter, especially near sharp corners where trajectory deviation becomes prominent.

The Tracking Error sub-figure of Figure 6 (bottom left) quantifies the evolution of position error over time. The corresponding Root Mean Square Error (RMSE) results are summarized in Table 1. MPPI achieves the lowest average tracking error across the entire task (RMSE = 0.0386 m), with peak deviation consistently under 2 cm. CEM-MPC performs comparably when $M = 30$ or $M = 7$ (RMSE = 0.0392 m and 0.0399 m, respectively), indicating that sufficient elite samples allow the method to approximate MPPI-level performance. However, when $M = 3$, RMSE increases sharply to 0.0493 m, reflecting instability in update direction due to inadequate elite representation.

Notably, RS-MPC produces an identical RMSE to CEM with $M = 3$ (0.0493 m), confirming that one-shot greedy selection tends to suffer from strong performance fluctuations, especially under dynamic and

Table 1: Tracking RMSE Comparison Between Controllers

Controller	RMSE (m)	Increase
MPPI	0.0386	—
CEM-MPC ($M = 30$)	0.0392	+1.6%
CEM-MPC ($M = 7$)	0.0399	+3.4%
CEM-MPC ($M = 3$)	0.0493	+27.7%
RS-MPC	0.0493	+27.7%

nonlinear trajectories.

The Position Components sub-figures of Figure 6 (bottom right) further shows the evolution of position along the X , Y , and Z axes. MPPI and CEM-MPC ($M = 30$) closely match the reference trajectory across all axes, particularly during aggressive climb, turning, and descent phases. RS-MPC, however, exhibits noticeable oscillations in the Y and Z directions, and abrupt changes near path corners, highlighting its limitations in modeling trajectory continuity.

Conclusion: From both the RMSE comparison and qualitative trajectory plots, it is evident that MPPI outperforms the other methods in terms of accuracy and stability. CEM-MPC can approach MPPI’s performance when a sufficiently large elite set is used, but it remains sensitive to parameter tuning. RS-MPC, although computationally efficient, fails to maintain robustness in multi-waypoint trajectory tasks due to its greedy selection mechanism and lack of global trajectory context. Overall, MPPI achieves the best performance by leveraging its path-integral-based optimization framework, which balances precision and smoothness while maintaining robustness across diverse task conditions.

4.1.2 MPPI Parameter Sensitivity Analysis: Horizon H and Sample Size N

Task Background: In sampling-based model predictive control methods such as MPPI, control performance is not only affected by system modeling and cost function design, but also highly dependent on internal hyperparameters. Among them, the prediction horizon H determines the length of the time window considered at each optimization step, influencing the controller’s ability to foresee future trajectory structures. Meanwhile, the number of samples N affects the robustness and convergence quality of the path integral estimate, directly impacting the stability and optimality of the resulting control input.

If the prediction horizon H is too short, the controller becomes myopic and cannot anticipate long-term objectives, leading to poor global performance. Conversely, overly large H may introduce state prediction errors and increase computation cost. Simi-

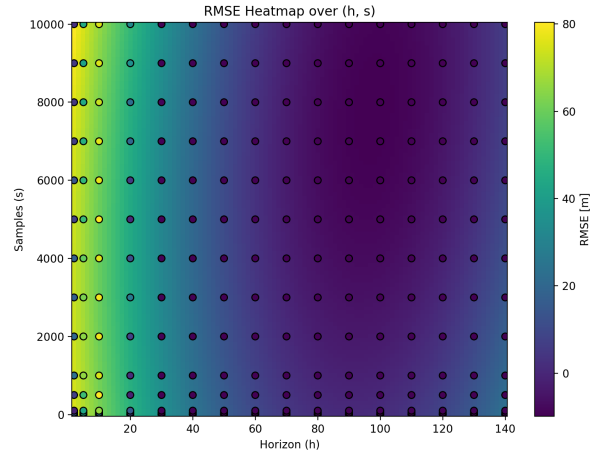


Figure 7: Heatmap of MPPI tracking error (RMSE) over different prediction horizons H and sample sizes N .

larly, a small sample size N results in noisy control estimates and degraded robustness, while too large N imposes high computational overhead. Hence, a systematic scan of these key hyperparameters is necessary to determine a reasonable trade-off configuration.

Experimental Setup: This experiment is conducted on the same multi-waypoint cubic trajectory task as in Section 4.1.1. The MPPI controller is evaluated across a 2D grid of parameter combinations in (H, N) space:

Prediction horizon $H \in \{2, 5, 10, \dots, 140\}$ (16 values);

Sample size $N \in \{2, 5, 20, 500, \dots, 10000\}$ (17 values);

Other parameters such as temperature λ , loss structure, and time step are kept constant to ensure fairness. For each (H, N) configuration, a full trajectory run is performed and the final RMSE of position tracking is recorded as the performance metric.

The results are presented in the form of a 2D heatmap, where color intensity indicates the magnitude of RMSE, darker regions represent lower error and better control performance.

Analysis and Discussion: Figure 7 shows the RMSE heatmap over the (H, N) parameter space. Several key patterns can be observed:

As shown in the heatmap, increasing the prediction horizon H leads to rapid error reduction. When $H < 30$, the RMSE remains high regardless of the number of samples N , indicating that the short prediction window fails to capture key trajectory features. As H increases into the range $[60, 100]$, the RMSE drops sharply and stabilizes, suggesting that the controller has sufficient foresight to plan consistent and accurate trajectories.

At a fixed H , increasing the number of samples N also improves performance in the range $N < 5000$.

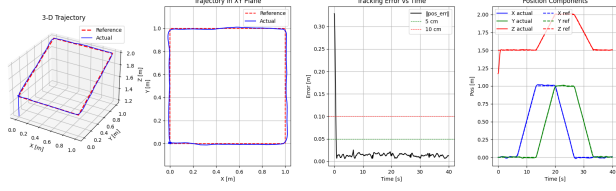


Figure 8: Trajectory tracking result using the optimal MPPI configuration ($H = 80$, $N = 5000$).

This reflects the benefit of denser sampling for path integral approximation. However, beyond $N = 5000$, the performance gains become marginal, showing diminishing returns despite the higher computational cost.

An optimal region emerges in the area $H = 80 \sim 100$, $N = 5000 \sim 10000$, where RMSE remains consistently low. This plateau indicates a robust operating zone in which the controller’s performance is relatively insensitive to small changes in hyperparameters, offering a favorable balance between accuracy and computational efficiency.

Final Configuration Selection: To balance control accuracy and computational efficiency, $H = 80$ and $N = 5000$ are chosen as the default MPPI configuration for all subsequent experiments. This setting lies within the stable performance plateau of the heatmap, achieving RMSE below 0.04m while keeping the per-step control delay about 10ms. As shown in Figure 8, the optimal MPPI configuration enables smooth and accurate execution of the trajectory with minimal overshoot or deviation. This trade-off makes it suitable for real-world deployment and complex path tracking tasks.

4.1.3 Loss-Term Ablation Study

Task Background: In MPPI controller design, the structure of the loss function is critical to shaping control behavior. Beyond the primary position tracking term, two auxiliary penalties are commonly introduced to enhance practical performance: a control input regularization term, which discourages overly large or abrupt commands to promote smooth trajectories, and an attitude constraint term, typically penalizing pitch and roll deviations to maintain stable flight posture and ensure the manipulator operates within a safe configuration envelope.

Experimental Setup: To evaluate the importance of these auxiliary terms, an ablation experiment is conducted by removing both the control input regularization and attitude penalty terms from the cost function, while keeping all other settings identical. The trajectory tracking task, initial conditions, and MPPI hyper-parameters (horizon $H = 80$, sample count $N = 5000$) are the same as in Section 4.1.1, remained unchanged for fair comparison.

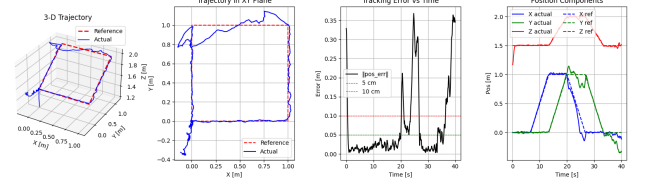


Figure 9: Trajectory tracking performance after removing control input and tilt penalty terms from the MPPI cost function.

Result and Analysis: Figure 9 illustrates the resulting trajectories under the ablated cost function. Although the system is still able to follow the overall path structure, noticeable oscillations and trajectory deviations are observed, particularly during turning and acceleration phases, indicating reduced smoothness and precision. The time-series tracking error plot further reveals that after $t = 20$ s, the control signal becomes increasingly erratic, leading to cumulative errors exceeding 20cm. This suggests the controller tends to select short-sighted but high-gain trajectories without the guidance of smoothness or posture constraints.

Conclusion: These results highlight the essential role of even low-weight auxiliary cost terms. Although their numerical contribution to the total loss may be small, their impact on overall control smoothness, stability, and robustness is significant. They not only shape the controller’s behavior in the local action space but also help compensate for finite-horizon prediction errors and dynamic uncertainties. Therefore, incorporating such regularization and posture-aware terms into the loss function is critical for achieving reliable, deployable trajectory tracking performance.

4.1.4 Disturbance-Rejection Benchmark

Task Background: To further validate the robustness of the MPPI controller in complex coupled aerial manipulation systems, a benchmark is designed to evaluate the system’s stability under actuator-induced disturbances. In aerial manipulation platforms, strong nonlinear coupling exists between the UAV and the onboard manipulator. Sudden or rapid manipulator motions, such as during grasping or swinging, can induce significant transient perturbations that compromise flight stability and trajectory tracking. Evaluating the controller’s response under such conditions is thus essential for real-world deployment.

Experimental Setup: Using the MPPI controller with the previously selected hyperparameters ($H = 80$, $N = 5000$) and the full cost function (including input and attitude penalties). The system starts with the manipulator at rest: all joint angles are initialized to zero and the end-effector is positioned at the target

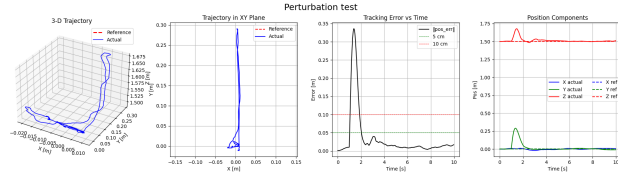


Figure 10: System response under a sudden high-speed manipulator disturbance (5 rad/s at $t = 1$ s).

location.

To simulate a realistic perturbation scenario, a sudden angular velocity input of 5 rad/s was applied to the manipulator joints at $t = 1$ s, mimicking high-speed actions such as object swinging or repositioning. This creates a sharp torque load on the UAV body and tests the controller’s rejection and recovery capabilities.

Results and Analysis: Figure 10 shows the system response during and after the disturbance, including the 3D trajectory, XY plane projection, time evolution of tracking error, and position components along the $X/Y/Z$ axes.

Upon the disturbance, the manipulator deviates sharply, leading to a sudden spike in position error to approximately 0.34 m. Y trajectories show abrupt deflection, while Z exhibits a brief overshoot due to coupling effects. Thanks to the MPPI controller’s rapid feedback and high-frequency sampling, the system absorbs the perturbation quickly. The peak deviation occurs around $t = 1.5$ s and the system recovers back to nominal tracking by $t = 2.0$ s. The total recovery time is under 1 second.

The position component plots confirm that all three axes return smoothly to the reference path without residual oscillation or overshoot. The tracking error plot exhibits a clear unimodal response, characteristic of a stable system with effective disturbance rejection and convergence.

Conclusion: This experiment demonstrates that the MPPI controller can handle strong transient perturbations (e.g., > 5 rad/s manipulator actuation) while maintaining overall system stability. It is capable of absorbing mechanical disturbances and regaining control within 1 second, showcasing its practical robustness for aerial manipulation tasks.

4.2 Real-World Experiments

Following comprehensive simulation-based evaluations, including trajectory tracking, parameter sensitivity analysis, disturbance rejection, and cost function ablation, the proposed control strategy is further validated on a real-world aerial manipulation platform. These experiments aim to assess the controller’s feasibility, stability, and robustness under physical conditions.

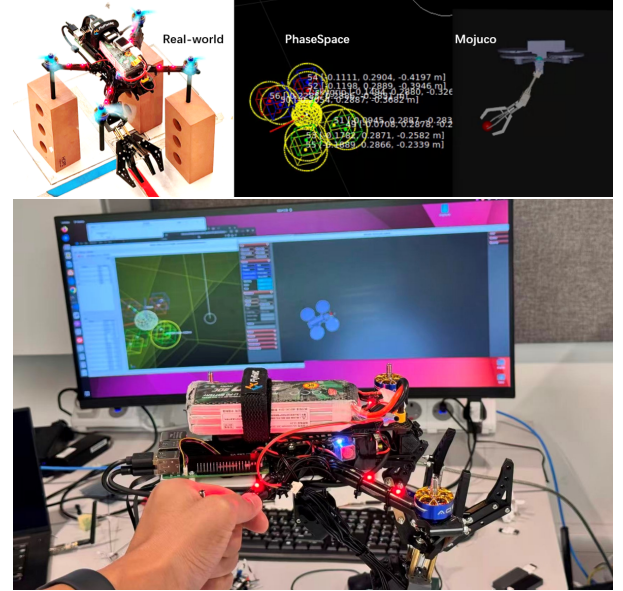


Figure 11: Overview of the aerial manipulation platform across different environments: Real-world hardware, PhaseSpace visualization, and MuJoCo simulation.

The experimental setup consists of a quadrotor UAV equipped with a lightweight 2-DoF robotic arm and a gripper. An external high-frequency optical motion capture system (PhaseSpace) provides accurate state estimation for closed-loop control. An overview of the hardware platform, the tracking visualization, and the MuJoCo simulation environment is illustrated in Figure 11, highlighting the integrated workflow from perception to control across both virtual and physical domains.

Onboard the UAV, a Raspberry Pi 5 serves as the main computational unit, running Paparazzi autopilot firmware alongside servo motor control logic. It is responsible for low-level motor actuation, sensor state integration, and serial communication.

The high-level control algorithm, namely, the MPPI controller, runs on a ground station PC equipped with an RTX 3060 GPU. The control loop operates at 18 ms intervals. Communication between the UAV and the ground station is handled via a custom UDP protocol, with a typical round-trip latency of approximately 5 ms.

System states are provided by the PhaseSpace motion capture system at 1000 Hz, with data streamed to the controller via a ROS2-based interface over Ethernet. This achieves an effective end-to-end latency of less than 30 μ s from sensing to perception. The overall data and control flow, including sensing, onboard execution, and offboard computation, is illustrated in Figure 12, which shows the real-time integration of all components across computational and physical layers. Together, this architecture forms a responsive, low-

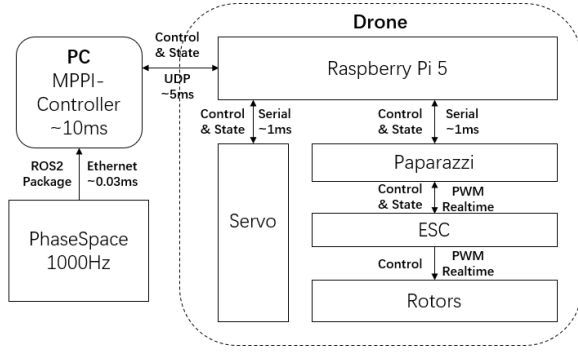


Figure 12: System architecture of the real-world aerial manipulation platform.

latency closed-loop control pipeline capable of high-frequency operation under real-world conditions.

Based on this hardware and software platform, three types of experiments were designed to evaluate the system’s dynamic behavior and modeling fidelity under real-world conditions.

1. **PWM Damping Model Ablation:** Assessing system response when the electrical damping model (used to compensate ESC nonlinearities) is disabled, to evaluate the impact of unmodeled actuation bias.
2. **Static Load Test on Robotic Arm:** Applying a 6 N static load to the end-effector and observing the resulting UAV body deformation and attitude deviations under passive compensation.
3. **Latency Analysis of the Closed-Loop System:** Quantifying the influence of total control latency on trajectory tracking accuracy, by injecting artificial delay into the communication loop.

These real-world experiments allow us to analyze the interaction between structural compliance and control policy performance, highlighting challenges that arise in hardware deployment and evaluating how well the proposed MPPI strategy can adapt to such imperfections.

4.2.1 Structural Deformation Analysis under Arm-End Load Disturbance

Task Background: In real-world aerial manipulation tasks, the robotic arm often induces structural perturbations and attitude deviations on the UAV platform, particularly during grasping, carrying, or object interaction. Static loads applied to the end-effector, such as grasped payloads, can be transmitted through the arm structure to the main platform, leading to both

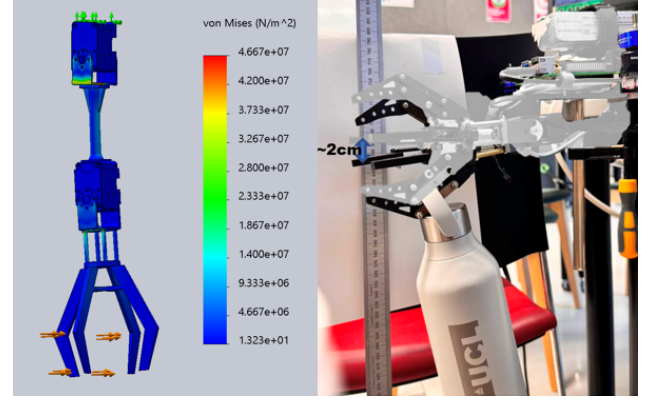


Figure 13: Structural analysis under 6 N end-effector load. Left: Finite element simulation showing von Mises stress distribution. Right: Real-world testing with mass suspended from gripper, displacement observed via ruler.

measurable deflections and attitude drift. This subsection investigates the structural response and platform stability under such static load conditions, using a combination of finite element analysis (FEA) and real-world measurements.

Experimental Setup: As shown in Figure 13, a 0.6 kg mass was suspended from the gripper, corresponding to a vertical static load of approximately 6 N. The quadrotor platform was rigidly fixed to a test bench, and downward deflection of the gripper was recorded. The resulting displacement was approximately 2 cm, primarily localized at the joint between the robotic arm body and its adapter mount, indicating a stress concentration and structural compliance in that region.

FEA Simulation: To systematically analyze stress distribution and potential deformation zones, a static FEA was conducted using the SolidWorks Simulation module. The base of the arm was set as a fixed constraint, and an equivalent 6 N lateral force was applied at the gripper tip, simulating the external load during object grasping. Material properties were set to match typical PLA plastic used in FDM 3D printing, considering its layer-wise Young’s modulus and yield strength for the most representative mechanical behavior.

Results and Analysis: As shown on the left of Figure 13, simulation results reveal that maximum stress occurs at the adapter interface between the upper servo mount and the gripper linkage, with peak von Mises stress reaching 42.0 MPa. Given that this region is aligned with the FDM printing layer orientation, and considering a conservative PLA yield strength of 50 ~ 60 MPa along the in-plane direction, the estimated safety factor lies between 1.2 and 1.3. This confirms that, under the current loading condition, the structure remains within safe operational margins and

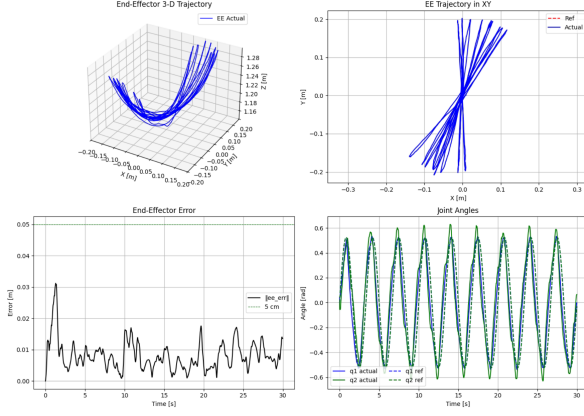


Figure 14: Trajectory tracking with PWMDD model enabled. End-effector motion is smooth and tightly bounded.

satisfies strength requirements.

However, despite sufficient strength, the structure exhibits low overall stiffness due to the lightweight plastic material. In the absence of active flight compensation, this leads to visible gripper sagging and notable attitude perturbations. Such deformation can adversely impact task precision, especially during dynamic flight conditions, potentially introducing oscillations or trajectory drift. Therefore, future control strategies should incorporate compensation mechanisms that consider structural deflection, such as real-time center-of-mass estimation and attitude feedback.

Conclusion: This experiment demonstrates that while the lightweight robotic arm satisfies strength constraints for aerial manipulation, its structural stiffness and connection integrity directly affect positioning accuracy. To reduce deformation under static loads, future designs could incorporate structural topology optimization, localized reinforcement ribs, or high-stiffness materials. Such improvements are essential to enhance platform stability and task precision in both static and dynamic aerial manipulation scenarios.

4.2.2 Ablation Study on the PWM-Based Damping and Drag Model (PWMDD)

Task Background: In the MPPI control framework, accurate actuator modeling is crucial for reliable prediction of future system trajectories. Conventional

control models often assume an ideal torque response, i.e., $\tau_{\text{eff}} \approx \tau_{\text{cmd}}$, where the motor is presumed to produce commanded torque instantaneously and linearly. However, in real-world implementations using PWM-driven servo motors without current feedback, this assumption breaks down due to various non-ideal behaviors, such as response delay, dynamic braking under overspeed conditions, and sudden torque drop under power loss (brake-on-disconnect behavior).

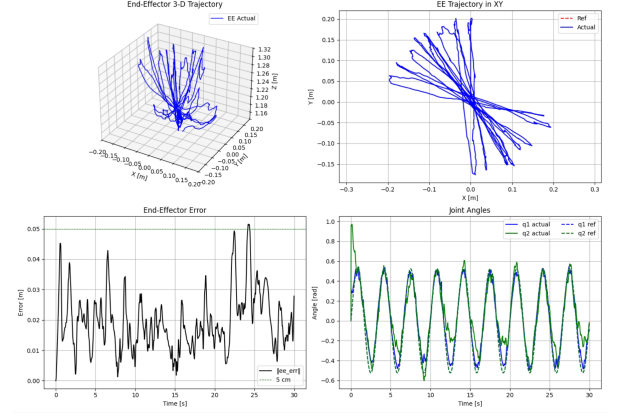


Figure 15: Trajectory tracking without PWMDD model. The system exhibits large oscillations and instability.

To improve fidelity between the control model and real actuator behavior, a PWM-based deviation damping (PWMDD) model is introduced into the joint dynamics. Specifically, the conventional torque-speed curve is augmented with a virtual damping term that is proportional to the angular velocity deviation $\Delta\omega = \omega_{\text{pwm}} - \omega_{\text{act}}$. This term captures the resistive and braking effects that occur due to the inherent limitations of open-loop PWM control, particularly when the commanded PWM frequency changes rapidly, but the actuator exhibits lag or overshoot due to inertia or friction. By explicitly modeling this behavior, the PWMDD model aligns the simulated joint response more closely with actual motion, especially under fast joint switching and transient conditions.

Experimental Setup: To evaluate the effect of this correction, a real-world ablation experiment is conducted on the physical two-joint robotic arm. The task consists of a series of rapid point-to-point joint movements. Control parameters, reference trajectories, and communication delays are kept fixed across trials. The only difference is whether the PWMDD model is activated in the actuator dynamics. Joint trajectories are recorded using onboard encoders, and the presence of oscillation, overshoot, and steady-state error is analyzed to assess the benefit of the damping augmentation. Results are discussed in terms of joint smoothness, tracking RMSE, and stability margins.

Results and Analysis: Figure 14 shows the tracking performance with PWMDD enabled. The end-effector exhibits smooth trajectories within a ± 10 cm envelope, and the mean tracking error remains below 2 cm. Joint angle profiles are also stable, with minimal overshoot or delay.

In contrast, Figure 15 presents the results without the PWMDD model. The system suffers from significant oscillations and instability: the end-effector deviates unpredictably from the target, with maximum

tracking errors exceeding 40 cm, and the joint angles show clear signs of lag and overshoot.

Conclusion: The experimental results clearly demonstrate the effectiveness of the PWMDD model in compensating for non-ideal characteristics of PWM-driven actuators. By bridging the gap between the controller’s internal prediction and the physical motor response, the model significantly enhances trajectory accuracy and system stability. As such, PWMDD forms an essential component of the actuator modeling pipeline for reliable aerial manipulation control.

4.2.3 Evaluation of High-Frequency Closed-Loop Latency

To assess the real-time feasibility of the proposed control system in a physical deployment scenario, a quantitative latency analysis was performed across the complete high-frequency closed-loop control pipeline. As illustrated in Figure 12, the system comprises five key stages: motion capture input, control computation, command transmission, servo actuation, and feedback acquisition.

Latency Analysis: To ensure the responsiveness and stability of the aerial manipulation system, we conduct a detailed breakdown of latency across the perception–computation–communication–execution pipeline:

- i) **Perception Latency:** A high-frequency PhaseSpace optical motion capture system is employed as the primary state estimation module, operating at 1000 Hz. The tracking data is transmitted directly to the main control PC via Gigabit Ethernet, bypassing middleware such as ROS or USB protocols. Measured frame acquisition and network propagation delay is approximately 0.03 ms per frame, rendering it negligible for real-time feedback control. The PhaseSpace system’s low jitter and high precision (0.5 mm) make it particularly well-suited for aggressive aerial tasks and fine-grained manipulator feedback.
- ii) **Control Computation:** The MPPI controller runs on a desktop-class GPU (RTX 3060) with a JAX-accelerated backend. Under the chosen configuration ($H = 80$, $N = 5000$), each control update, covering trajectory rollout, cost evaluation, and action extraction, takes around 10, ms, supporting a 55, Hz loop rate. A grid search over (H, N) pairs (Figure 7) reveals that while increasing samples and horizon improves tracking accuracy, the benefit plateaus beyond moderate values. The selected setup thus strikes a balance between computational cost and control precision.

- iii) **Communication Latency:** Control commands are transmitted from the PC to the onboard Raspberry Pi 5 via UDP over a dedicated local Ethernet interface. This connection introduces a one-way latency of 2–3 ms with sub-millisecond jitter. Compared to wireless or serial-based communication links, this wired Ethernet setup significantly reduces latency variance, which is critical for high-frequency receding-horizon control. Periodic heartbeat packets are also used to maintain synchronization and detect link dropouts.

- iv) **Onboard Execution Latency:** The Raspberry Pi communicates with the UAV flight controller and Dynamixel servos via separate UART buses. A synchronized group polling mechanism is implemented to aggregate actuator command dispatch and sensor feedback within each control cycle. Empirical measurements indicate that joint state polling takes approximately 0.05 ms, and servo command transmission consumes around 1.5 ms. Due to continuous polling rather than interrupt-based communication, the system achieves low-latency response with minimal buffering delay. Real-time thread priorities are also configured to ensure consistent execution timing under multi-process workloads.

End-to-End Latency Breakdown: The total delay across the control pipeline can be estimated as follows:

$$\begin{aligned}
 \text{Total Delay} &\approx 0.03 \text{ ms (motion capture)} \\
 &\quad + 10 \text{ ms (controller)} \\
 &\quad + 5 \text{ ms (network)} \\
 &\quad + 1.5 \text{ ms (UART)} \\
 &= 16.5 \text{ ms}
 \end{aligned}$$

Hardware Platform Comparison: To assess the computational feasibility of deploying the MPPI controller in real-world applications, its execution latency was benchmarked across multiple hardware platforms with varying compute capabilities. The results are presented in Table 2. On CPU-only platforms, such as the AMD Ryzen 5 5600X and Intel i7-12700K, the average computation time per control cycle was measured at 200–300, ms. This latency significantly exceeds the control frequency required for stable real-time operation, rendering such configurations unsuitable for embedded deployment.

In contrast, when executed on a desktop-class GPU (NVIDIA RTX 3060), the controller achieves an average runtime of approximately 10, ms per iteration. This level of performance enables control frequencies

Table 2: MPPI Runtime Comparison on Different Hardware Platforms

Platform	Device Type	Time (ms)
RTX 3060	GPU (Desktop)	10
RTX 4080	GPU (Laptop)	16
Intel i7-12700K	CPU (Desktop)	200
AMD R5 5600X	CPU (Desktop)	300

exceeding 90 Hz, which aligns with the real-time demands of high-agility aerial manipulation. A laptop-class GPU (RTX 4080 Laptop) demonstrated slightly higher latency (16ms), likely due to power management and thermal throttling constraints, though still within usable bounds for moderately aggressive control tasks.

These results highlight the importance of leveraging parallel hardware acceleration, particularly GPU computing, to meet the computational demands of sampling-based whole-body control algorithms such as MPPI, especially in high-dimensional and tightly coupled robotic systems.

Conclusion: The proposed architecture, featuring onboard real-time feedback and external high-speed computation, successfully balances real-time performance and engineering feasibility. The achieved end-to-end latency of approximately 16.5 ms fully supports 50–100 Hz control frequencies, and the low jitter in both the motion capture and actuation stages ensures rapid disturbance rejection and attitude correction.

Final Remarks: Following extensive simulation and physical testing, the proposed MPPI-based whole-body control framework has demonstrated its feasibility on a real aerial manipulation platform. The experimental results verify strong performance in static precision control, disturbance compensation from manipulator motion, and structural deflection handling under load. Furthermore, the fidelity of the dynamic model and inclusion of the PWMDD actuator model proved critical for achieving simulation-to-real transfer. These findings lay a solid foundation for future extensions toward dynamic in-flight manipulation tasks.

5 CONCLUSION

This paper presents a systematic study of a self-developed quadrotor-manipulator system, covering the entire process from mechanism design and dynamic modeling to optimal control. We first designed a lightweight two-DoF manipulator and validated it through finite element analysis, then calibrated the thrust and torque constants of the propellers via thrust stand experiments. Based on these results, we established a unified dynamic model that incorporates center-of-gravity offsets and actuator character-

istics. On this foundation, we implemented a GPU-accelerated Model Predictive Path Integral (MPPI) controller and, for the first time in such systems, introduced a PWM Deviation Damping (PWMDD) model to improve servo stability under open-loop PWM control. Simulation results demonstrate that MPPI outperforms traditional Cross-Entropy Method MPC (CEM-MPC) and Random Shooting MPC (RS-MPC) methods in trajectory-tracking accuracy and disturbance rejection, while PWMDD effectively suppresses oscillations in the manipulator joints. Both simulations and real-world experiments further verify the platform’s ability to follow multi-waypoint trajectories, recover from attitude disturbances, and maintain stable performance under a 6 N payload, highlighting strong control performance and consistency between simulation and physical testing.

Despite the promising performance achieved, several limitations remain. First, the manipulator has only two degrees of freedom, restricting end-effector flexibility and limiting task complexity, which constrains the system’s overall manipulation capability. Second, the MPPI controller depends heavily on manual expertise in cost function design and parameter tuning, lacking an automated parameter adjustment mechanism that would improve its generalization across tasks. Third, the control framework still relies on external perception (PhaseSpace) and host-side decision making, falling short of achieving fully onboard closed-loop control, which reduces deployment flexibility. Furthermore, although low-latency closed-loop control has been achieved, the current Wi-Fi-based UDP communication still exhibits significant latency and jitter that warrant further optimization.

For future work, we plan to advance in several directions: (i) hardware-wise, extending manipulator DoFs and integrating end-effectors such as grippers and tactile modules to enable more complex tasks; (ii) algorithmically, exploring MPPI variants that integrate reinforcement learning and self-supervised cost adjustment to enhance generalization and adaptability; (iii) perception-wise, incorporating lightweight visual SLAM or depth sensing modules to handle unstructured environments; and (iv) system-wise, building a fully onboard control pipeline by deploying the entire control stack on lightweight, high-performance embedded platforms (e.g., NVIDIA Jetson or ARM edge devices), thereby achieving a flight-operation system with high-precision control, real-time responsiveness, and integrated environmental perception.

In summary, the system architecture, modeling approach, and control framework developed in this work establish a solid engineering foundation for future research on high-DoF, strongly coupled aerial manipulation systems, while also providing valuable insights

and pathways toward multi-modal perception fusion, generalized control strategies, and fully autonomous aerial operation tasks.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Valerio Modugno, for his invaluable guidance, support, and encouragement throughout the course of this project. His insights in control theory, system integration, and overall project direction were instrumental in shaping both the research scope and implementation path.

Special thanks also go to Mr. Bonot Gautam, who provided extensive support with the laboratory infrastructure and experimental resources, and to Mr. Danny Steadman and Mr. Simon Gosling from the Bartlett Fabrication Lab for his practical assistance during 3D printing and soldering phases.

I am particularly grateful to my fellow student Chenzhe Jin, with whom I shared countless hours of debugging, testing, and collaborative development; our mutual support was an essential part of this project’s progress.

Finally, I would like to thank Stephanie Jat, a junior colleague working on related Paparazzi flight control systems, for her contributions and engaging technical discussions.

Appendix: Project Resources and Repositories

This project is publicly hosted on GitHub at the following address:

- **Main Repository:** <https://github.com/Headmaster218/SkyGrip>

Code Structure

- The `main` branch contains the primary implementation of the quadrotor–manipulator system, including simulation, control, and real-world integration code.
- The `compare-test` branch contains additional scripts and experimental configurations used for performance comparisons in the “Experiments and Results” section.
- The core platform integration and testing code is located at:
 - https://github.com/Headmaster218/SkyGrip/blob/main/Sim'n'Real/sbmpc/examples/quadrotor_arm_test.py

Media and Reports

All figures, diagrams, and experimental videos referenced in the report can be found at:

- **Reports Directory:** <https://github.com/Headmaster218/SkyGrip/tree/main/Reports>
- **Medias Directory:** <https://github.com/Headmaster218/SkyGrip/tree/main/Medias>

These include rendered CAD models, simulation screenshots, and video footage from real-world testing.

External Dependencies

This project incorporates and extends the **SBMPC** library for GPU-accelerated sampling-based model predictive control:

- **SBMPC Repository:** <https://github.com/tombelv/sbmpc>

This library is embedded in the project and used to accelerate both the simulation and control processes on GPU-compatible hardware.

References

- [1] Tommaso Belvedere et al. “Feedback-MPPI: Fast Sampling-Based MPC via Rollout Differentiation–Adios low-level controllers”. In: *arXiv preprint arXiv:2506.14855* (2025).
- [2] I. Boukabou, N. Kaabouch, and D. Rupanetti. “Cybersecurity Challenges in UAV Systems: IEMI Attacks Targeting Inertial Measurement Units”. In: *Drones* 8.738 (2024). DOI: 10.3390/drones8120738.
- [3] James Bradbury et al. *JAX: Composable Transformations of Python+NumPy Programs (v0.3.13)*. <https://github.com/google/jax>. accessed 16 Aug. 2025. 2018.
- [4] Lukas Brunke et al. “Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning”. In: *Annual Review of Control, Robotics, and Autonomous Systems* 5 (2021), pp. 411–438. DOI: 10.1146/annurev-control-042920-020211.
- [5] B. B. Carlos et al. “An Efficient Real-Time NMPC for Quadrotor Position Control under Communication Time-Delay”. In: *Proc. 16th Int. Conf. on Control, Automation, Robotics and Vision (ICARCV)*. Shenzhen, China, 2020.
- [6] Min Chen, Qing Wu, and Chuanwen Jiang. “Disturbance-observer-based robust synchronization control of uncertain chaotic systems”. In: *Nonlinear Dynamics* 70.4 (2012), pp. 2421–2432.

- [7] Moritz Diehl, Hans Joachim Ferreau, and Jan Haverbeke. “Efficient Numerical Methods for Nonlinear MPC and Moving Horizon Estimation”. In: *Nonlinear Model Predictive Control* (2009), pp. 391–417. DOI: 10.1007/978-3-642-01094-1_32.
- [8] Wei Dong et al. “High-performance trajectory tracking control of a quadrotor with disturbance observer”. In: *Sensors and Actuators A: Physical* 211 (2014), pp. 67–77.
- [9] Mohamed Elhesasy et al. “Non-Linear Model Predictive Control Using CasADi Package for Trajectory Tracking of Quadrotor”. In: *Energies* 16.5 (2023), p. 2143. DOI: 10.3390/en16052143.
- [10] Mohamed Fanni and Ahmed Khalifa. “A new 6-dof quadrotor manipulation system: Design, kinematics, dynamics, and control”. In: *IEEE/ASME Transactions on Mechatronics* 22.3 (2017), pp. 1315–1326.
- [11] Filippo Forte et al. “Impedance control of an aerial manipulator”. In: *2012 American Control Conference (ACC)*. IEEE. 2012, pp. 3839–3844.
- [12] Gil Geun-oh. “Adaptive Neural Network Controller for Quadrotor with Two Degree of Freedom Robotic Arm”. PhD thesis. Seoul National University Graduate School, 2017.
- [13] Farhad A. Goodarzi, Dongjun Lee, and Taeyoung Lee. “Geometric control of a quadrotor uav transporting a payload connected via flexible cable”. In: *International Journal of Control, Automation and Systems* 13.6 (2015), pp. 1486–1498.
- [14] Miguel E. Guerrero-Sánchez et al. “Swing-attenuation for a quadrotor transporting a cable-suspended payload”. In: *ISA transactions* 68 (2017), pp. 433–449.
- [15] Daniel Hanover et al. “Performance, Precision, and Payloads: Adaptive Nonlinear MPC for Quadrotors”. In: *IEEE Robotics and Automation Letters* 7.3 (2022), pp. 690–697. DOI: 10.1109/LRA.2021.3139942.
- [16] Zifei Jiang. “Nonlinear Control of Aerial Manipulators”. In: (2024).
- [17] Mina Kamel, Michael Burri, and Roland Siegwart. “Linear vs Nonlinear MPC for Trajectory Tracking Applied to Rotary Wing Micro Aerial Vehicles”. In: *IFAC-PapersOnLine* 50.1 (2017), pp. 3463–3469. DOI: 10.1016/j.ifacol.2017.08.1559.
- [18] A. Khalifa, S. M. Shaaban, and A. Khalifa. “Hybrid Disturbance Observer and Fuzzy Logic Controller for a New Aerial Manipulation System”. In: *Frontiers in Robotics and AI* 12 (2025). DOI: 10.3389/frobt.2025.1528415.
- [19] Ahmed Khalifa, Mohamed Fanni, and Alaa Khalifa. “Quadrotor Manipulation System: Development of a Robust Contact Force Estimation and Impedance Control Scheme Based on DOB and FTRLs”. In: *arXiv preprint arXiv:1904.00008* (2019).
- [20] Seungjoo Kim, Seung Choi, and H. Jin Kim. “Aerial manipulation using a quadrotor with a two dof robotic arm”. In: *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2013, pp. 4990–4995.
- [21] Christopher Korpela, Matko Orsag, and Paul Oh. “Towards valve turning using a dual-arm aerial manipulator”. In: *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2014, pp. 3411–3416.
- [22] H. Li et al. “Adaptive Neural Network Backstepping Control Method for Aerial Manipulator Based on Variable Inertia Parameter Modeling”. In: *Journal of the Franklin Institute* 361.7 (2024), p. 106733. DOI: 10.1016/j.jfranklin.2024.106733.
- [23] Shihua Li et al. *Disturbance observer-based control: methods and applications*. CRC Press, 2014.
- [24] Vincenzo Lippiello and Fabio Ruggiero. “Cartesian impedance control of a UAV with a robotic arm”. In: *10th IFAC Symposium on Robot Control*. IFAC. 2012, pp. 704–709.
- [25] Steve Macenski et al. “Robot Operating System 2: Design, Architecture, and Uses in the Wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074.
- [26] Robert Mahony, Vijay Kumar, and Peter Corke. “Multirotor Aerial Vehicles: Modeling, Estimation, and Control”. In: *IEEE Robotics & Automation Magazine* 19.3 (2012), pp. 20–32. DOI: 10.1109/MRA.2012.2206474.
- [27] G. Malczyk et al. “Multi-Directional Interaction Force Control with an Aerial Manipulator Under External Disturbances”. In: *Autonomous Robots* 47 (2023), pp. 1325–1343. DOI: 10.1007/s10514-023-10128-2.

- [28] Lorenz Meier, Daniel Honegger, and Marc Pollefeys. “PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms”. In: *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*. Seattle, USA, 2015.
- [29] Daniel Mellinger et al. “Design, Modeling, Estimation and Control for Aerial Grasping and Manipulation”. In: *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. San Francisco, CA, USA: IEEE, Sept. 2011, pp. 2668–2673. DOI: 10.1109/IROS.2011.6094871.
- [30] Xiangdong Meng, Yuqing He, and Jianda Han. “Survey on aerial manipulator: System, modeling, and control”. In: *Robotica* 38.7 (2020), pp. 1288–1317.
- [31] M. Minařík et al. “Model Predictive Path Integral Control for Agile Unmanned Aerial Vehicles”. In: *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*. Abu Dhabi, UAE, 2024, pp. 13144–13151.
- [32] Tiago Nascimento and Martin Saska. “Embedded Fast Nonlinear Model Predictive Control for Micro Aerial Vehicles”. In: *Journal of Intelligent & Robotic Systems* 103 (2021), p. 74. DOI: 10.1007/s10846-021-01367-0.
- [33] NVIDIA. *Jetson Orin Nano Developer Kit Getting Started Guide*. <https://developer.nvidia.com/embedded/learn/jetson-orin-nano-devkit-user-guide/index.html>. accessed 16 Aug. 2025. 2025.
- [34] Matko Orsag, Christopher Korpela, and Paul Oh. “Modeling and control of mm-uav: Mobile manipulating unmanned aerial vehicle”. In: *Journal of Intelligent & Robotic Systems* 69.1-4 (2013), pp. 227–240.
- [35] J. Pravitra et al. “L1-Adaptive MPPI Architecture for Robust and Agile Control of Multirotors”. In: *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*. Las Vegas, USA, 2020, pp. 7661–7666. DOI: 10.1109/IROS45743.2020.9341015.
- [36] M. A. Rafique and A. F. Lynch. “Adaptive Backstepping Control of an Unmanned Aerial Manipulator”. In: *Preprints* (2025). Preprint, DOI:10.20944/preprints202508.1963.v1.
- [37] Reuven Y. Rubinstein. “The Cross-Entropy Method for Combinatorial and Continuous Optimization”. In: *Methodology and Computing in Applied Probability* 1.2 (1999), pp. 127–190.
- [38] Fabio Ruggiero et al. “Impedance control of VTOL UAVs with a momentum-based external generalized forces estimator”. In: *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2014, pp. 2093–2099.
- [39] M. A. Sadi, A. Jamali, and A. M. N. Abang Kamaruddin. “Optimizing UAV Performance in Turbulent Environments Using Cascaded Model Predictive Control Algorithm and Pixhawk Hardware”. In: *Journal of the Brazilian Society of Mechanical Sciences and Engineering* 47 (2025), pp. 1–26. DOI: 10.1007/s40430-025-05234-1.
- [40] Emre Sariyildiz et al. “A nonlinear stability analysis for the robust position control problem of robot manipulators via disturbance observer”. In: *2015 IEEE International Conference on Mechatronics (ICM)*. IEEE. 2015, pp. 28–33.
- [41] Aditya Sehgal et al. “WireFlie: A Novel Obstacle-Overcoming Mechanism for Autonomous Transmission Line Inspection Drones”. In: *IEEE Robotics and Automation Letters* (2025).
- [42] Koushil Sreenath, Taeyoung Lee, and Vijay Kumar. “Geometric Control and Differential Flatness of a Quadrotor UAV with a Cable-Suspended Load”. In: *Proc. 52nd IEEE Conf. on Decision and Control (CDC)*. Firenze, Italy, 2013, pp. 2269–2274. DOI: 10.1109/CDC.2013.6760172.
- [43] Jakob Thumm and Matthias Althoff. “Provably Safe Deep Reinforcement Learning for Robotic Manipulation in Human Environments”. In: *arXiv preprint arXiv:2205.06311* (2022).
- [44] Tomas Tomic and Sami Haddadin. “A unified framework for external wrench estimation, interaction control and collision reflexes for flying robots”. In: *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2014, pp. 4197–4204.
- [45] G. Turrisi et al. “On the Benefits of GPU Sample-Based Stochastic Predictive Controllers for Legged Locomotion”. In: *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*. Abu Dhabi, UAE, 2024, pp. 13757–13764.
- [46] P. C. N. Verheijen et al. “Multilevel Parallel GPU Implementation of SQP Solvers for Nonlinear MPC”. In: *Proc. European Control Conference (ECC)*. Stockholm, Sweden, 2024, pp. 2292–2298. DOI: 10.23919/ECC64448.2024.10636502.

- [47] Robin Verschueren et al. “acados—A Modular Open-Source Framework for Fast Embedded Optimal Control”. In: *Mathematical Programming Computation* 14 (2022), pp. 147–183. DOI: 10.1007/s12532-021-00208-0.
- [48] Bogdan Vlahov et al. “MPPI-Generic: A CUDA Library for Stochastic Trajectory Optimization”. In: *arXiv preprint arXiv:2409.07563* (2024).
- [49] Boris Vlahov et al. *MPPI-Generic: A CUDA Library for Stochastic Optimization*. arXiv:2409.07563. 2024.
- [50] Grady Williams, Andrew Aldrich, and Evangelos Theodorou. “Model predictive path integral control using covariance variable importance sampling”. In: *arXiv preprint arXiv:1509.01149* (2015).
- [51] Grady Williams, Andrew Aldrich, and Evangelos A. Theodorou. “Model Predictive Path Integral Control: From Theory to Parallel Computation”. In: *Journal of Guidance, Control, and Dynamics* 40.2 (2017), pp. 344–357. DOI: 10.2514/1.G001921.
- [52] Grady Williams et al. “Aggressive Driving with Model Predictive Path Integral Control”. In: *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*. Stockholm, Sweden, 2016, pp. 1433–1440. DOI: 10.1109/ICRA.2016.7487277.
- [53] Wenli Xiao et al. “Safe Deep Policy Adaptation”. In: *arXiv preprint arXiv:2310.08602* (2023).
- [54] Guangyu Zhang et al. “Robust Control of An Aerial Manipulator Based on A Variable Inertia Parameters Model”. In: (2024). Focuses on dynamic coupling and inertia variations affecting control.
- [55] Z. Zhang et al. “Strong Electromagnetic Interference and Protection in UAVs”. In: *Electronics* 13.393 (2024). DOI: 10.3390/electronics13020393.
- [56] S. Zhao et al. “Integral Sliding Mode Control-Based Anti-Disturbance Controller for Unmanned Aerial Manipulators”. In: *Aerospace* 12.9 (2025), p. 764. DOI: 10.3390/aerospace12090764.